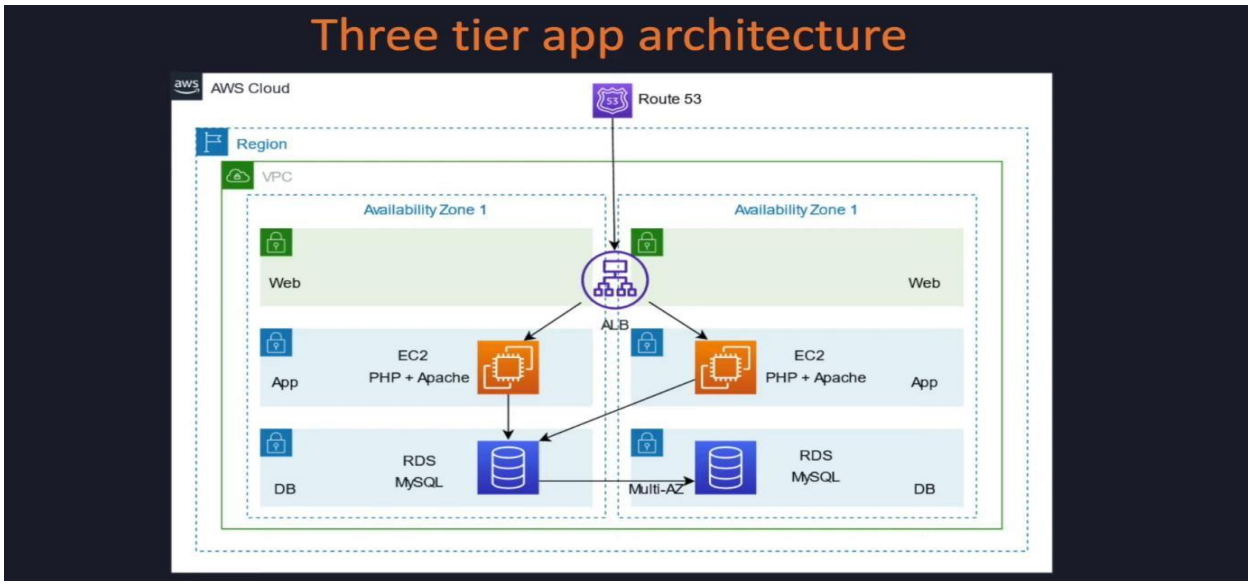
AWS Challenge

(Designing Robust Web Application on AWS)



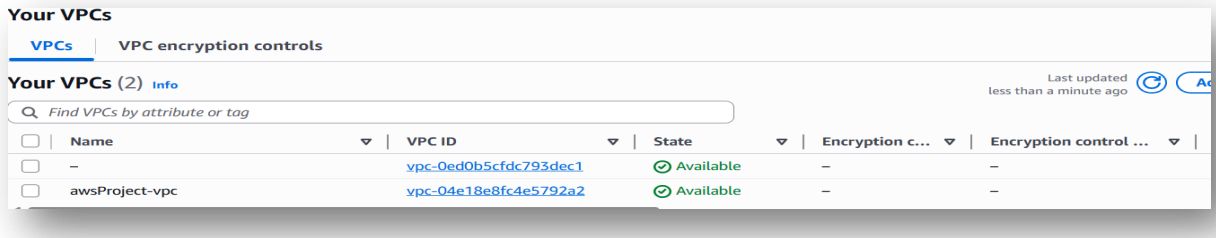
Summary:

This AWS architecture diagram shows a reliable web application built with three parts. Users find the application through Route 53, a DNS service. The application is protected in a virtual private network (VPC) on AWS, spread across different availability zones to ensure it keeps working even if one zone fails. The user-facing part of the application uses multiple servers(EC2 Instances) for better performance, with an Application Load Balancer (ALB) sharing the traffic evenly. The business logic is handled in another part, possibly using a database service. Data is securely stored in a separate part, using a database service RDS. This setup provides a strong base for web applications on AWS, with the flexibility to add more services for advanced features.

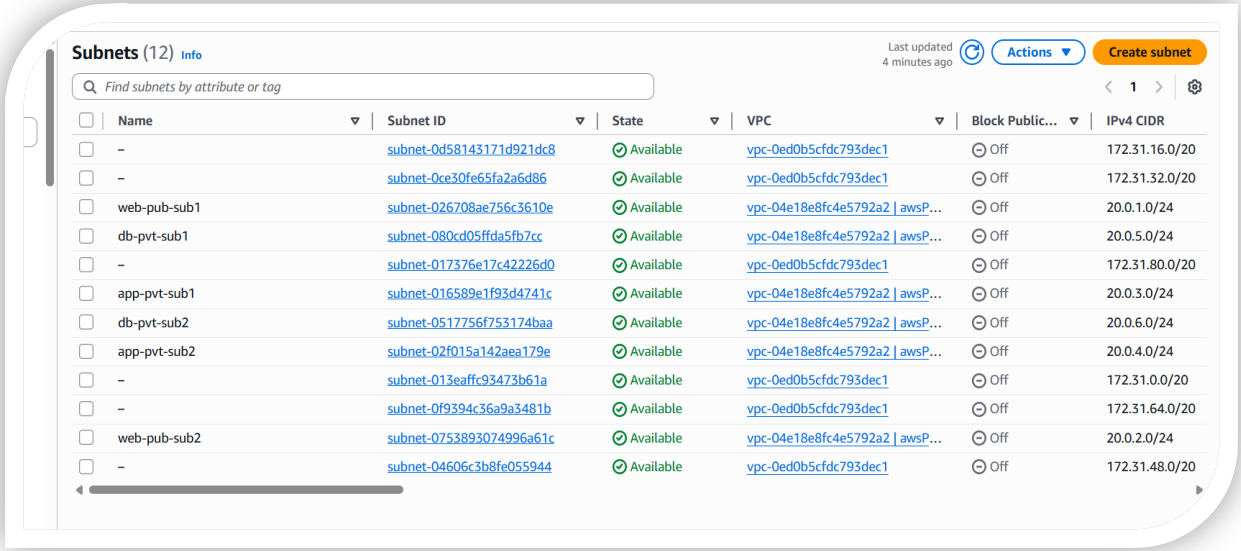
Lets proceed:

Part 1

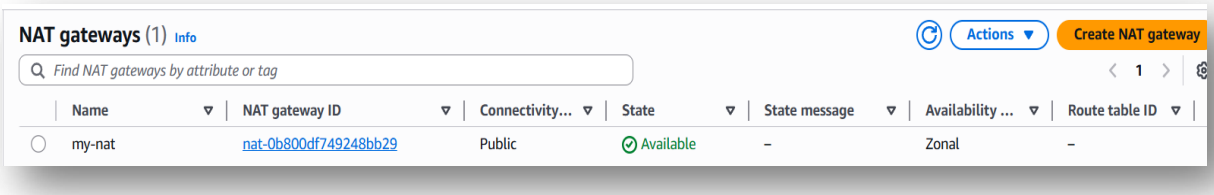
- Create a VPC named awsProject-vpc with a CIDR block of 20.0.0.0/20.



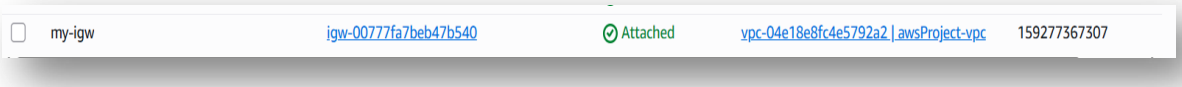
- Create 2 public subnets:
  - web-pub-sub1 with CIDR block 20.0.1.0/24 in us-east-1
  - web-pub-sub2 with CIDR block 20.0.2.0/24 in us-east-1b
- Create 2 private subnets for applications:
  - app-pvt-sub1 with CIDR block 20.0.3.0/24 in us-east-1a
  - app-pvt-sub2 with CIDR block 20.0.4.0/24 in us-east-1b
- Create 2 private subnets for databases:
  - db-pvt-sub1 with CIDR block 20.0.5.0/24 in us-east-1a
  - db-pvt-sub2 with CIDR block 20.0.6.0/24 in us-east-1b



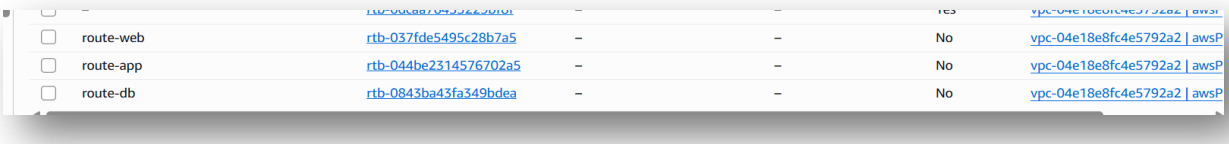
- Create a NAT Gateway named my-nat, provide the subnet as web-pub-sub1, and allocate an Elastic IP.



- Create an Internet Gateway named my-igw and attach it to the VPC awsProject-vpc.



- Create 3 route tables named route-web, route-app, and route-db.
- Associate the subnets with the route tables as follows:
  - Select route-web and associate it with the subnets web-pub-sub1 and web-pub sub2.
  - Select route-app and associate it with the subnets app-pvt-sub1 and app-pvt sub2.
  - Select route-db and associate it with the subnets db-pvt-sub1 and db-pvt-sub2.



Now add the routes

Select route-web, go to routes and click edit routes, add route select the Destination as 0.0.0.0/0,

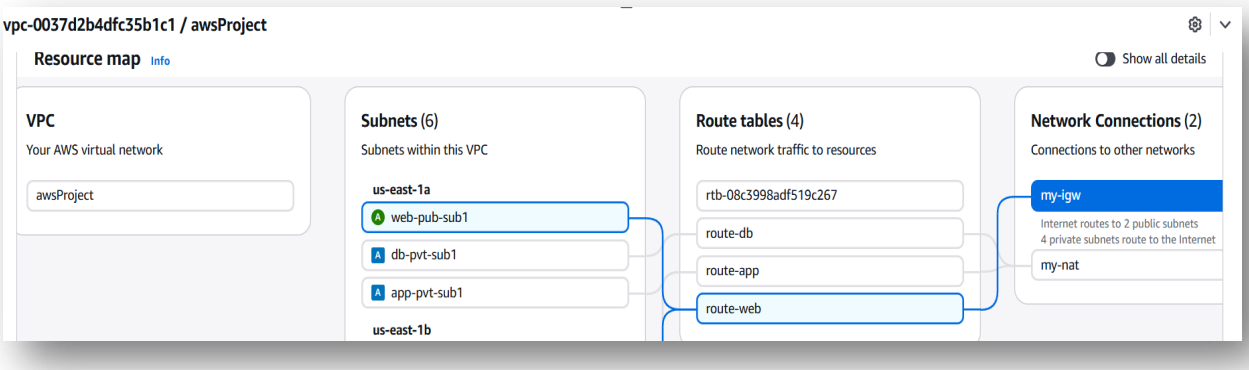
Target Internet Gateway(my-igw) *and click on save changes*

Select route-app, go to routes and click edit routes, add route select the Destination as 0.0.0.0/0,

Target NAT Gateway(my-nat) *and click on save changes*

Select route-db, go to routes and click edit routes, add route select the Destination as 0.0.0.0/0,

Target NAT Gateway(my-nat) *and click on save changes*



Part 2

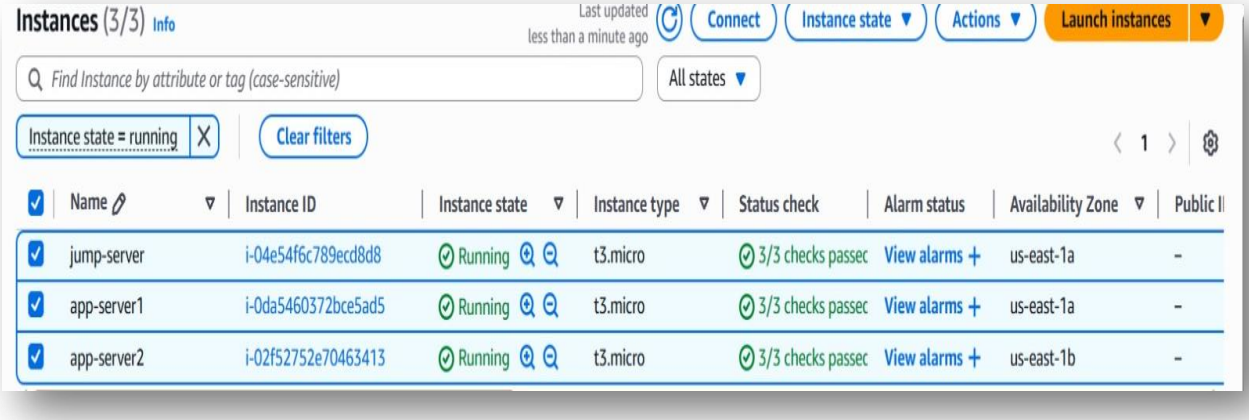
Create 3 EC2 instances in which 1 in public subnet with publicIP enable which acts as Jump server or bastion host and 2 private subnet with publicIP disable in which we will download phpMyAdmin and apache server

1) Launch an Instance with name jump-server, AMI (Amazon Linux), Instance type (t2.micro), Create a new keypair as (projectkey) , click on edit button on right side of Network settings select vpc(awsProject-vpc), Subnet (web-pub-sub1), Auto-assign IP (Enable), Create security group [Security group name (jump-sg)], allow port SSH (22) and HTTP (80) now Launch instance

Create a security group as alb-sg and allow port HTTP (80)

2) Launch an Instance with name app-server1, AMI (Amazon Linux), Instance type (t2.micro), Select the keypair as (projectkey) , click on edit button on right side of Network settings select vpc (awsProject-vpc), Subnet (app-pvt-sub1), Auto-assign IP (Disable), Create security group [Security group name (app-sg)], allow port SSH (22) and Click on Add security group rule select Type (All traffic ,) Source type (Custom) and Source (here select your [alb-sg] you created) now Launch instance

3) Launch an Instance with name app-server2, AMI (Amazon Linux), Instance type (t2.micro), Select the keypair as (projectkey) , click on edit button on right side of Network settings select vpc (awsProject-vpc), Subnet (app-pvt-sub2), Auto-assign IP (Disable), Select existing security group as (app-sg), now Launch instance



Copy the private key in your jump-server instance

>> scp -i <your key> <yourkey path> ec2-user@<private ip>:~

>> chmod 400 <your key>

Now you can ssh into your app-server1 and app-server2 instance

ssh into your jump-server and your private key should be present in home directory

Now ssh into app-server1 and run the below commands

Update the system

>> sudo yum update -y

Install PHP 8.2

>> sudo dnf install php8.2

>> sudo yum install php8.2-mysqlnd

Install Apache Web Server

>> sudo yum install -y httpd

Start and Enable Apache

>> sudo systemctl start httpd

>> sudo systemctl enable httpd

>> sudo systemctl is-enabled httpd

Add User to Apache Group

>> sudo usermod -a -G apache ec2-user

Change Ownership and Permissions for Web Directory

>> sudo chown -R ec2-user:apache /var/www

>> sudo chmod 2775 /var/www && find /var/www -type d -exec sudo chmod 2775 {} \;

>> find /var/www -type f -exec sudo chmod 0664 {} \;

Install Additional PHP Modules

>> sudo yum install php-mbstring php-xml -y

>> sudo yum install php-fpm

Restart Apache and PHP-FPM

>> sudo systemctl restart httpd

>> sudo systemctl restart php-fpm

Download and Set Up phpMyAdmin

>> cd /var/www/html

>> wget https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz

>> mkdir phpMyAdmin && tar -xvzf phpMyAdmin-latest-all-languages.tar.gz -C phpMyAdmin --strip components 1

>> rm phpMyAdmin-latest-all-languages.tar.gz

Create a Test Page and Test the Server

>> echo "PHP server 2" > /var/www/html/index.html

>> curl

http://localhost

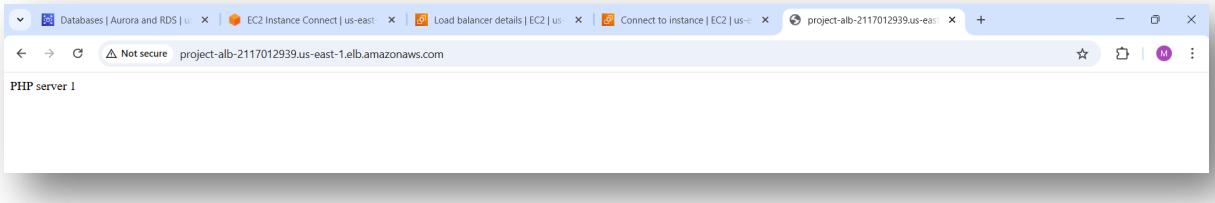
As you have downloaded Apache and phpMyAdmin to access them, you now need to:
Create the Target groups:

Go to target groups, create a target group with target type (Instance), name (app-tg), Port HTTP 80, IP address type IPv4, VPC (awsProject-vpc), Protocol version HTTP1, Health checks as HTTP, Health check path / and click on next, now select the app-server1 and app-server2 in Available instances, ports 80 click Include as pending below and click on Create target group.

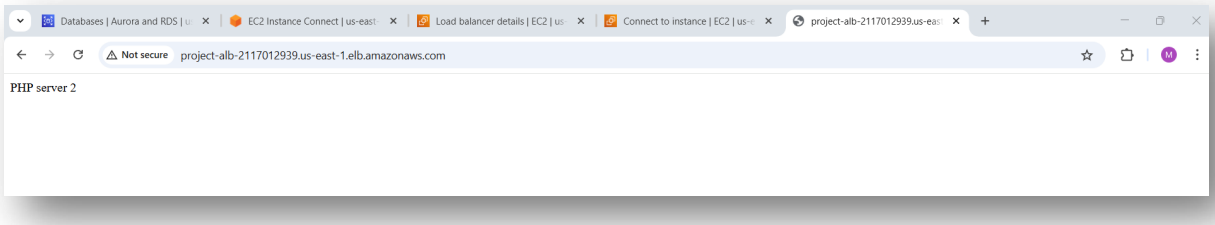
Now go to Load Balancer:

Create Application Load Balancer with name project-alb, Scheme (Internet-facing), Load balancer IP (IPv4), VPC (awsProject-vpc), Mappings us-east-a1, us-east-1b, select the security groups as (alb-sg), Listener [Protocol (HTTP), Port 80, Target group (app-tg)] Click on Create load balancer

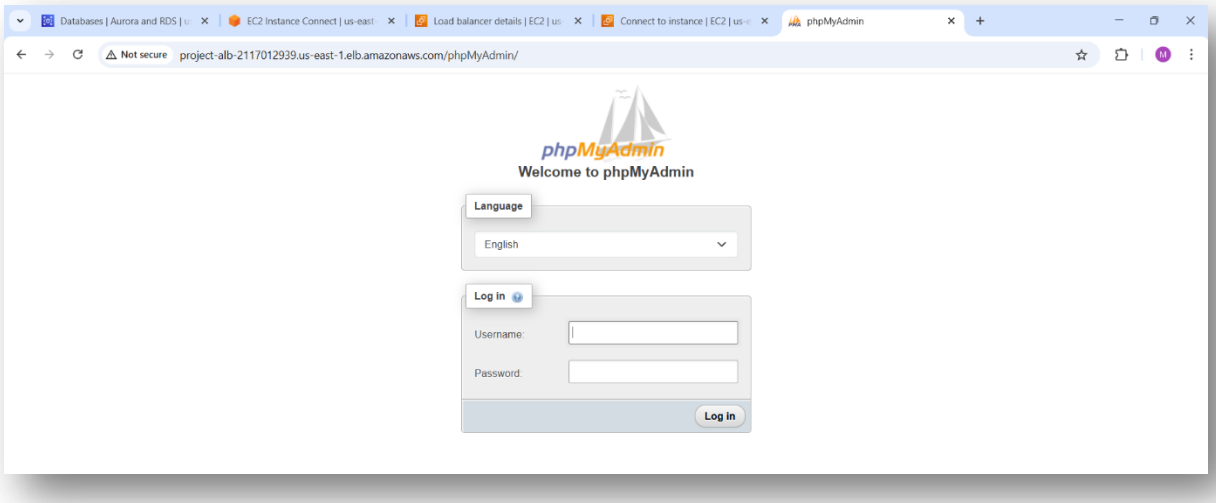
Now take the load balancer DNS and past it on chrome browser you should see
PHP Server 1 and when you refresh the page it should show PHP Server 2 that means your load balancing is working as expected



Refrest the page:



And when you write phpMyAdmin after your domain name it should navigate to



Part 3

Go to RDS

Select Subnet groups, Create a Subnet group and name it as db-subnetgroup, give description as db-subnetgroup, VPC (awsProject-vpc), Select the subnets that were created for db as (db-pvt-sub1 and db-pvt-sub2) and click on create

Now go to Databases,Click on Create database, select Stancerd create, Engine options

(MySQL), Templates (Free tier), Settings DB instance identifier (mydbproject), Master username (admin), Credentials management (Self managed),

Master password (admin), Confirm password (admin), Instance configuration

DB instance class (db.t3.micro), Storage Click on Storage autoscaling and

uncheck the (Enable storage autoscaling) Connectivity Compute resource (Don't

connect to an EC2 compute resource), VPC (awsProject-vpc), DB subnet group (db

subnetgroup), Public access (No), VPC security group (Create new)… New VPC

security group name (db-sg), Availability Zone (us-east-1a), Click on Create

database.

Now go to security group (db-sg) click on edit inbound rules, Add rule, select Type

(CustomIP),Port 3306, Source select the (app-sg) and remove the above

MYSQL/Aurora rule and save changes

Now copy the Endpoint of RDS cluster and ssh into app-server1 instance and app-server2

instance

>>cd /var/www/html/phpMyAdmin

>> mv config.sample.inc.php config.inc.php

>> nano config.inc.php

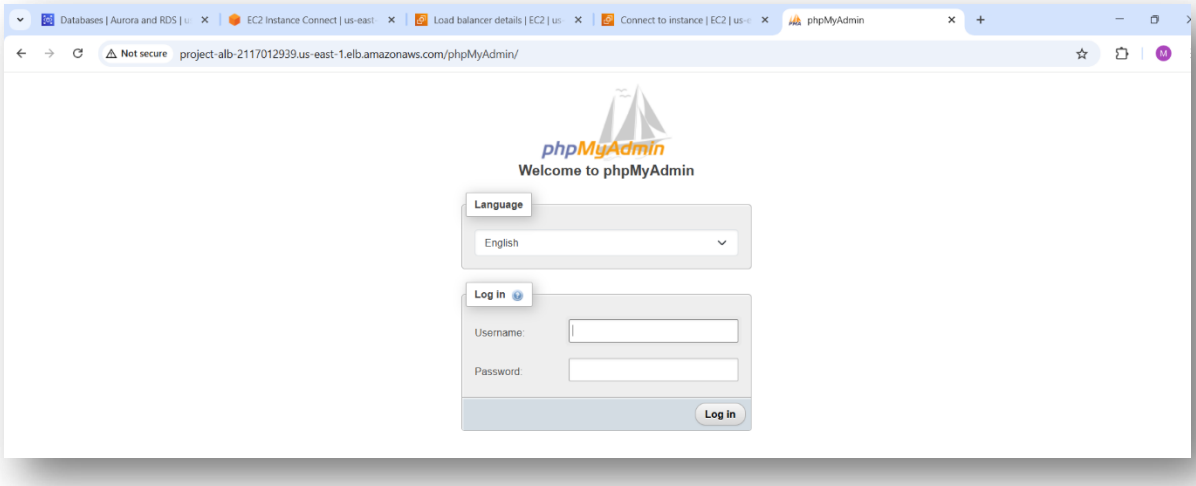
And search for host and remove 'localhost' and replace it with cluster endpoint, now save the file

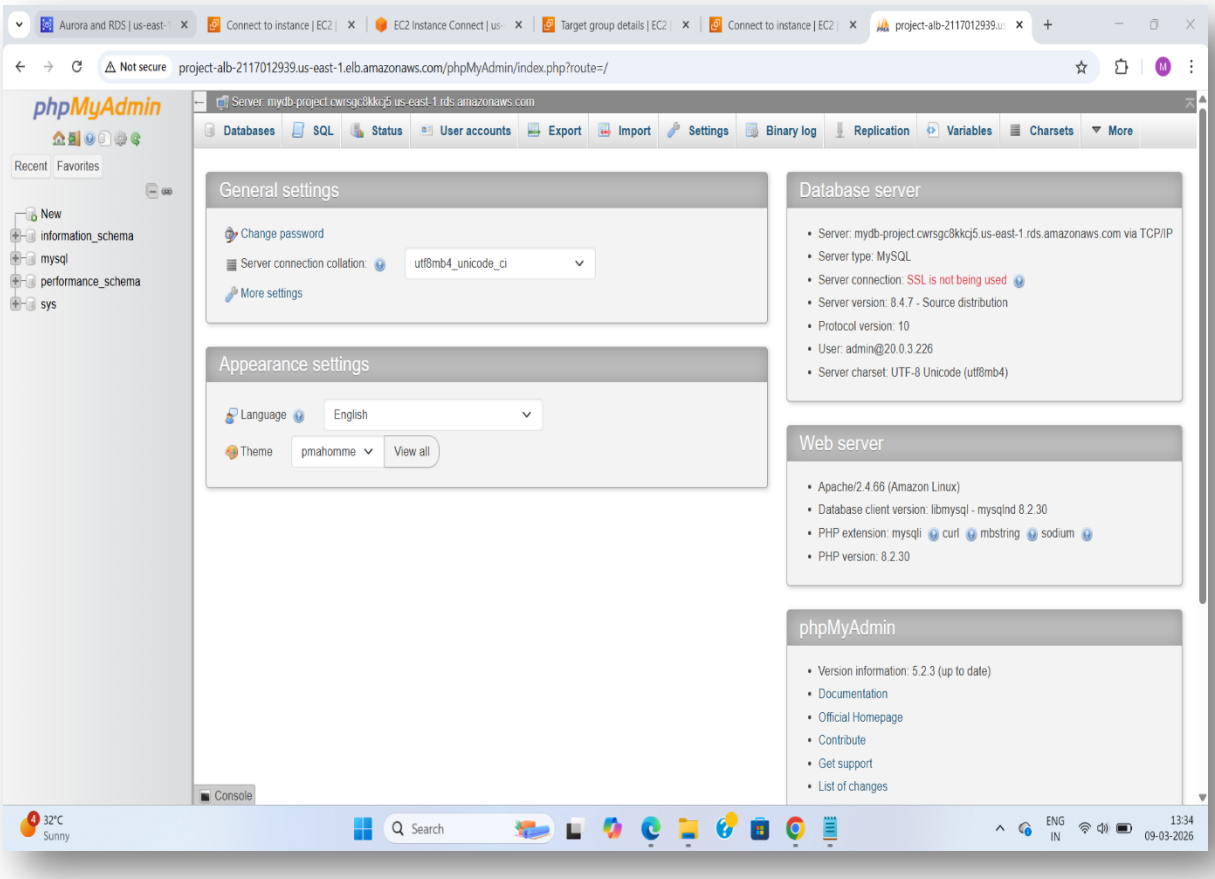
One last thing , go to target groups, select the (app-tg), go to Attributes and click on edit, scroll down

and click on Turn on stickiness (Load balancer generated cookie) and save changes

Go to chrome paste the loadbalancer DNS with /myPhpAdmin at the last and give the username, and

password you will be able to login.





Submitted by: MEHARUNISSA

E-mail: zmd35211@gmail.com